

Common Web Attacks

Unit 11 — XSS, Command Injection & File Inclusion

Every bug in this unit has the same root cause — **untrusted input** — and the same cure: **validate, encode, and isolate it.**

Learning objectives

By the end of this unit you can:

- **Explain** why **unvalidated input** is the root cause behind XSS, command injection, and file inclusion.
- **Distinguish** the three types of **XSS**: reflected, stored, DOM-based.
- **Demonstrate** reflected and stored XSS in DVWA and **describe** its defenses (output encoding + CSP).
- **Demonstrate** a **command injection** and describe its defenses (safe APIs, allow-list).
- **Demonstrate** an **LFI** with directory traversal and **distinguish** it from RFI.
- **Observe** how raising DVWA's **security level** (stronger validation) blocks each attack.
- **Pair every attack with its defense**, in writing.

The one root cause

What happens when an app trusts input it should not trust?

- The user types something → the app uses it without checking → the input changes what the app *does*.
- XSS, command injection, file inclusion are the **same mistake** in three places.

The shared cure: validate input, encode output, isolate dangerous operations.

Cross-Site Scripting (XSS)

XSS lets an attacker run their JavaScript in someone else's browser.

- The app takes input and puts it on a page **without encoding it**.
- The browser can't tell the attacker's `<script>` from the site's own code — so it **runs** it.
- Impact: stolen cookies/sessions, defaced pages, keylogging.

```
<!-- lab-only proof payload -->  
<script>alert(1)</script>
```

Three types of XSS

Type	What happens	Who it hits
Reflected	Input bounces straight back in <i>this</i> response (via URL/param)	Whoever opens the crafted link
Stored	Input is saved in the app and served later	Every future visitor
DOM-based	Happens in the browser's own JavaScript	Whoever runs that JS

Stored XSS is the most dangerous — it hits people who never consented.



Defense: stopping XSS

1. Output encoding — convert characters so the browser shows them as **text**, not code:

```
<script> → &lt;script&gt;
```

The browser now *displays* `<script>` instead of running it.

2. Content Security Policy (CSP) — a response header that tells the browser **which scripts are allowed to run**, so injected scripts are blocked.

Encode on **output**; validate/sanitize on **input**; add **CSP** as a backstop. Defense-in-depth.

Command Injection

The app passes your input into an operating-system command.

- A "ping" tool runs `ping <your input>` in the **shell**.
- Add a chaining character and the shell runs **your** command too:

```
# normal input
127.0.0.1
# lab-only injection
127.0.0.1; whoami
```

- Output reveals the user the web server runs as (e.g., `www-data`) — proof your input reached the shell.



Defense: stopping command injection

- **Don't call the shell at all** — use a safe library/function for the task.
- **Safe API / parameterization** — input is handled as **data**, never assembled into a command string.
- **Validate & allow-list** — accept *only* valid characters (for an IP, only digits and dots).
- **Least privilege** — the web server account can do as little as possible.

Blocking "bad" characters (a blacklist) can be bypassed. **Allow-listing known-good is stronger.**

File Inclusion (LFI vs RFI)

The app loads a file chosen by user input. A URL like `?page=home` becomes a problem when you ask for something else.

	Loads	Danger
LFI (Local)	A file already on the server	Read secrets like <code>/etc/passwd</code>
RFI (Remote)	A file from a remote attacker URL	Worse — can run attacker code

```
?page=../../../../etc/passwd
```

- `../` = **directory traversal**: climb up out of the intended folder.



Defense: stopping file inclusion

- **Validate / allow-list** the requested file — accept only known-good names.
- **Never build a file path directly from user input.**
- **Disable remote includes:** `allow_url_include = off` (kills RFI).

Same pattern again: don't trust input, restrict to known-good, isolate the dangerous operation.

The payoff: raise the security level

- DVWA's **Security Level** (low → medium → high) turns up **input validation**.
- Re-run the *same* payloads at a higher level → most or all are **blocked**.

If your attack "stops working," that **is the lesson**, not a bug.

- Medium often uses naive filtering (e.g., stripping `../` once) — bypassable.
- High applies proper validation/encoding — the real fix.

Ethics & Authorization

These attacks, against a real site, **steal other people's sessions** (XSS), **run commands on someone else's server** (command injection), or **read files that aren't yours** (file inclusion).

Everything here runs against **DVWA — built on purpose to be broken — in an isolated lab ONLY**. Never the school network, never a real site. No "I was just curious" exception under the **CFAA**.

We pair every attack with its defense: the goal is to become the person who **fixes** these bugs.

Key vocabulary

Term	Meaning
Untrusted input	Any data from the user/outside — must be checked
Input validation	Checking type, length, format before trusting
Allow-list	Permit only known-good values (safer than blocking bad)
XSS (reflected/stored/DOM)	Attacker's JS runs in another user's browser
Output encoding	Show characters as text, not code
CSP	Header limiting which scripts may run
Command injection	Input reaches an OS shell command
Safe API / parameterization	Input treated as data, never as a command
LFI / RFI	Load a local file / a remote attacker file



Lab launch

Platform: DVWA (Damn Vulnerable Web Application)

- Via TryHackMe "DVWA" AttackBox (simplest) or an **isolated** local image. Log in `admin` / `password`, set Security to **Low**.
- Trigger: **reflected XSS → stored XSS → command injection → LFI**.
- Then **raise the security level** and watch each attack get blocked.
- For **every** attack, record the matching **defense** in your Attack ↔ Defense chart.

Scope statement first: *"I am authorized to test only DVWA inside the isolated class lab."*

Recap

- One root cause: **untrusted input**. One cure: **validate, encode, isolate**.
- **XSS** → output encoding + CSP.
- **Command injection** → safe APIs + allow-list input.
- **File inclusion (LFI/RFI)** → validate/allow-list paths + disable remote includes.
- Stronger **input validation** (higher security level) blocks all three.

Exit ticket & discussion

1. What single **root cause** connects all three attacks, and what single idea **cures** them?
2. How does **output encoding** stop XSS? Name one defense for command injection.
3. **Discuss:** Why is *stored* XSS considered more dangerous than *reflected* XSS?

Next — Unit 12: SQL Injection